Group Anagrams

AlgoExpert · Medium · Strings

The problem

- Group words that are anagrams of each other (key = sorted letters).

Intuition

- Anagrams share the same sorted letters, which makes a perfect hash key.

Brute force vs optimal

- Compare every pair with a frequency check → $O(w^2 \cdot n)$
- Sorted-letters key in a hash map → $O(w \cdot n \log n)$
- Count-array key → $O(w \cdot n)$

Approach

- 1. Sort each word's letters
- 2. Group words by that key
- 3. Return the groups

Worked example

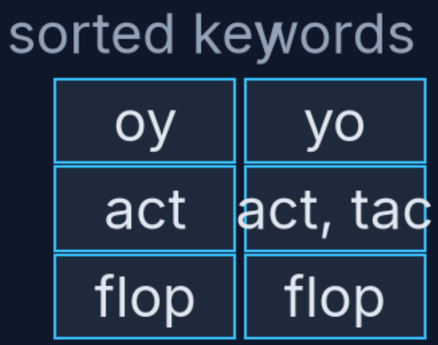
sorted keywords

oy	yo

```
'yo' → key 'oy'
```

'yo' → 'oy'

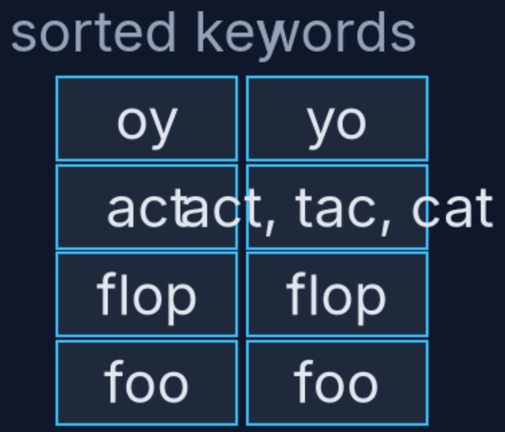
Worked example




```
'tac' → key 'act'
```

'tac' → 'act'

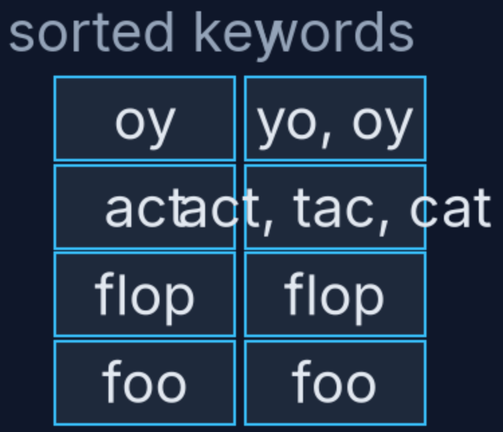
Worked example




```
'cat' → key 'act'
```

'cat' → 'act'

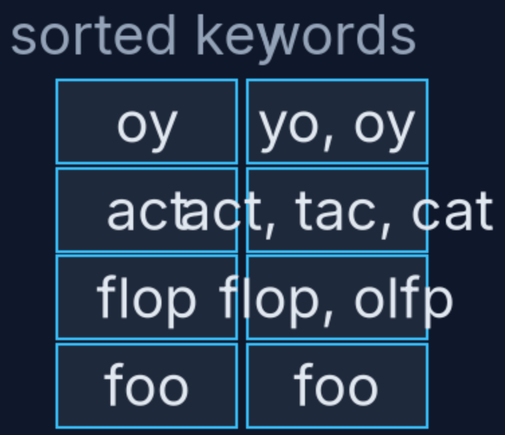
Worked example




```
'oy' → key 'oy'
```

'oy' → 'oy'

Worked example




```
'olfp' → key 'flop'
```

'olfp' → 'flop'

Java solution

```java
public class GroupAnagrams {
    public static List<List<String>> groupAnagrams(List<String> words) {
        Map<String, List<String>> groups = new LinkedHashMap<>();
        for (String w : words) {
            char[] c = w.toCharArray();
            Arrays.sort(c);
            groups.computeIfAbsent(new String(c), k -> new ArrayList<>()).add(w);
        }
        return new ArrayList<>(groups.values());
    }
}
```

Complexity

- Time: O(w * n log n)
- Space: O(w * n)

Edge cases & pitfalls

- Empty list → []
- Keep insertion order if required
- Case sensitivity

Interview tips & follow-ups

- Mention the count-key optimisation

Recap

- Anagrams share the same sorted letters, which makes a perfect hash key.
- Complexity: O(w * n log n)